Rheinisch-Westfälische Technische Hochschule Aachen
Alessandro Berti

User Guide For "Conformance Predictions"

Group 4:
Richard Schuppener, Cem Colak

Contents

1 Introduction **3**

2 Installation Guide **3**

2.1 Excerpt: Installation Guide from Requirements Engineering Document 3

3 User Guide **4**

3.1 Step 1: Event Log and Normative Model 4

3.1.1 Has model? 4

3.1.2 Event log path 4

3.1.3 Option A: Loading a Pre-Built Model 4

3.1.4 Option B: Discovering a Model 5

3.2 Step 2: Normative Model Visualization (Optional) 5

3.3 Step 3: Prediction Target 6

3.4 Step 4: Feature Type 7

3.4.1 Selecting case attributes 8

3.5 Step 4.1: Prefix mode 8

3.5.1 Classification targets (process outcome / future deviations) 8

3.5.2 Remaining time 9

3.6 Step 4.2: Train/Test Split 9

3.7 Step 5: ML Algorithm and Hyperparameters 10

3.7.1 Classification algorithms (process outcome / future deviations) 10

3.7.2 Regression algorithms (remaining time) 10

3.7.3 Hyperparameters 10

3.8 Step 6: Decision Tree Visualization and Refinement 11

3.9 Step 7: Evaluation Results 12

3.10 Step 8: Saving Evaluation Visuals 13

3.11 Step 9: Predictions on New Data 13

3.12 Step 10: Runtime Report 13

3.13 Known Limitations 14

1 Introduction

In this document we will provide a user guide for the "Conformance Predictions" project. The user guide is split into several sections. First, we will provide a guide for the installation of our project, followed by a guide for the usage of our software artifact.

2 Installation Guide

In our past Requirements Engineering document we have already outlined the Installation procedure for our project. In the following section we will provide the guide for completeness.

2.1 Excerpt: Installation Guide from Requirements Engineering Document

To use our **Python script**, the following set-up steps are required. First, the user should check whether all prerequisites are met: Our project requires a working **Python** installation of version *3.14.4* (currently the latest available Python version where all required packages are supported). Also, a proper installation of **Git** is required.

First, the repository needs to be cloned:

```
git clone
https://git.rwth-aachen.de/schuppenerr/conformance-predictions.git
```

Then, we suggest entering the cloned directory and creating a virtual environment for the Python packages:

```
python -m venv .venv
```

Alternatively (not recommended), this step can be skipped, and the required packages can be globally installed on the PC.

The next step is to install all packages from the *requirements.txt* file:

```
pip install -r requirements.txt
```

Now, the script can be launched via the terminal from the directory containing *main.py*:

```
python main.py
```

Note that this aims only to roughly sketch the installation process and neither depicts correct BPMN nor comprehensively describes the script setup.

3 User Guide

3.1 Step 1: Event Log and Normative Model

3.1.1 Has model?

`Do you have a normative process model? (y/N):`

Answer `y` if you have a pre-built model file ready, or `n` (the default) to let the tool discover a model automatically from the event log.

3.1.2 Event log path

`Enter the path to the event log (XES/CSV):`

Enter the absolute or relative path to your event log. Two file formats are accepted:

- **XES** (`.xes`) — the standard process mining log format; loaded directly via PM4Py.
- **CSV** (`.csv`) — the file must contain columns with exactly the names `case:concept:name`, `concept:name`, and `time:timestamp`. Any other column naming will cause a loading error.

3.1.3 Option A: Loading a Pre-Built Model

If you answered `y` in the first prompt:

`Enter the path to the normative model (Petri net/BPMN):`

Three formats are accepted:

- **Petri net** (`.pnml`) — loaded directly.
- **BPMN** (`.bpmn`) — converted to a Petri net internally before use.
- **Process tree** (`.ptml`) — converted to a Petri net internally before use.

Any other file extension will cause an error.

3.1.4 Option B: Discovering a Model

If you answered `n`, the tool prints:

```
Model discovery will be used.

Which discovery algorithm should be used?
1. Inductive Miner (recommended)
2. Alpha Miner
3. Genetic Miner
Select option (1-3):
```

Option 1 — Inductive Miner is the recommended choice for most logs. Two additional parameters are requested:

```
Noise threshold (default 0.0):
Disable fall-throughs? (y/N):
```

The noise threshold (a value between 0.0 and 1.0) controls how aggressively infrequent behavior is filtered out before model construction. A value of 0.0 keeps all behavior; higher values produce simpler models. Press `Enter` to accept the defaults.

Option 2 — Alpha Miner has no configurable parameters. The tool will confirm this and continue:

```
Alpha Miner has no configurable parameters.
```

Option 3 — Genetic Miner requests six parameters:

```
Population size (default 500):
Elitism rate (default 0.01):
Crossover rate (default 1.0):
Mutation rate (default 0.01):
Generations (default 100):
Elitism min sample (default 5):
```

Press `Enter` on any field to keep its default value.

3.2 Step 2: Normative Model Visualization (Optional)

After the model is loaded or discovered, the tool asks:

```
Do you want a visualization of the [discovered/provided]
normative model saved (s) or directly displayed (d) or not (N)?
(s/d/N):
```

We recommend you to visualize the model at least once in the beginning to get a better understanding of the process structure. Especially for the prediction outcome target (option 2 in Step 3), it is important to know which activities signify the different outcomes. The visualization can be skipped in later runs once you are familiar with the model.

- **s — Save:** the model is rendered and written to a file. Two further prompts follow. First, choose the representation format:

  ```
  Choose model representation for visualization:
  1. Petri Net
  2. BPMN
  3. Process Tree
  Select option (1-3):
  ```

  Then provide the output file path:

  ```
  Enter the path to save the visualization (e.g., model.png):
  ```

- **d — Display:** the same representation prompt is shown, then the figure(png) is opened in an interactive window. No file is written.
- **N — Skip:** no visualization is produced, and the pipeline continues immediately.

3.3 Step 3: Prediction Target

```
What should be predicted?
1. Future conformance deviations
2. Process outcome
3. Remaining time
Select option (1-3):
```

Option 1 — Future conformance deviations predicts whether a case will exhibit a further deviation from the normative model. The label is binary: 0 means no (further) deviation is expected, 1 means a deviation is expected. After selecting this option, one additional prompt appears:

```
Deviation cost threshold T (default 0):
```

A case is counted as *deviant* when its real alignment cost — the number of genuine log-only and visible model-only moves, with silent routing excluded — exceeds the threshold `T`. With `T=0` (the default) any real deviation makes the case deviant; raising `T` tolerates minor deviations before a case is labelled deviant. Press `Enter` to keep `T=0`. *Fully implemented.*

Option 2 — Process outcome predicts which of several predefined outcomes a case will reach. After selecting this option, the following prompt appears:

```
Define the target outcome:
Enter activity names that signify a outcome (one per line).
Type 'done' when finished.
>
```

Enter each outcome activity name **exactly as it appears in the event log**, including capitalization, one name per line. Type `done` and press `Enter` when finished. Each case is labelled with the outcome activity that appears in it, provided exactly one of the specified activities is present. Cases containing none or more than one of the specified activities receive the label `uncertain_outcome`. These uncertain cases indicate, that the chosen outcome activities are not given. *Fully implemented.*

Option 3 — Remaining time predicts, from a partial trace, how much time (in seconds) remains until the case completes. This is a regression target rather than a classification target, so it leads to the regression algorithm menu in Step 5. Because the remaining time of a completed case is always zero, this target **requires prefixing** (see Step 4.1); the "no prefixing" option is therefore not offered for it. *Fully implemented.*

3.4 Step 4: Feature Type

```
What should the prediction be based upon?
1. Plain trace (control-flow)
2. Trace and alignment-based diagnostics
3. Case features
4. Mixture of all available information
Select option (1-4):
```

Option 1 — Control-flow uses a bag-of-activities representation. For each case the feature vector contains the count of each activity observed in the log. *Fully implemented.*

Option 2 — Alignment-based diagnostics runs optimal alignment between the log and the model and encodes, for each case, the counts of synchronous moves, log-only moves, and model-only moves per activity. This is

computationally more expensive than option 1 for large logs. *Fully implemented.*

Option 3 — Case features uses static case-level attributes (e.g. a patient's age or admission department) stored in the event log. *Fully implemented.*

Option 4 — Hybrid combines all of the above: control-flow counts, alignment-based diagnostics, and the selected case attributes. *Fully implemented.*

3.4.1 Selecting case attributes

If you chose option 3 or option 4, the tool lists the case-level attributes it found in the log and asks you to select which to use:

```
Available case-level attributes:
  1. <attribute>  (type=..., unique=..., e.g. ...)
  2. ...
Select attributes (comma-separated numbers, or 'all'):
```

Enter the numbers of the attributes you want, separated by commas, or type `all` (or press `Enter`) to use every listed attribute. If the log contains no case-level attributes, the tool prints a notice and continues without case features.

3.5 Step 4.1: Prefix mode

A prefix is the first part of a trace. Training on prefixes lets the model predict from a *partial* case, as it would be used in practice. How prefixes are formed is controlled by the prefix-mode prompt.

3.5.1 Classification targets (process outcome / future deviations)

For the two classification targets the tool asks:

```
Prefix mode?
  [Enter] none      (default - one row per case, cheapest; no per-prefix alignme
  1)      exactly k (one prefix per case at length k)
  2)      varying 1..k (pooled - prefixes of length 1..k)
Select (Enter/1/2):
```

- **none** (press `Enter`) — one row per case using the full trace. This is the cheapest path: at most one alignment per case.

- **exactly k** (enter `1`) — one prefix per case, cut at a fixed length. You are then asked:

  ```
  prefix length k:
  ```

- **varying 1..k** (enter `2`) — pooled prefixes of every length from 1 up to `k`. You are then asked for the maximum length and the step:

  ```
  max length k (blank = median case length):
  step (default 1):
  ```

Note: when the feature type is *pure case attributes* (option 3 in Step 4), no prefix is needed — the prompt is skipped and the model runs on full traces by design.

3.5.2 Remaining time

Remaining time always needs a partial trace, so the "none" option is not offered. The prompt instead reads:

```
Prefix mode (required for remaining time)?
  [Enter] varying 1..median (N)  (pooled - prefixes of length 1..k)
  1)      exactly k          (one prefix per case at length k)
Select (Enter/1):
```

Here `N` is the median case length of your log, used as the default upper bound. Pressing `Enter` pools prefixes of length 1 up to the median; entering `1` asks for a single fixed length `k` instead.

3.6 Step 4.2: Train/Test Split

```
Enter a split value for the train- / test-split (default=0.7):
```

The log is split into a training and a test set using a temporal split by case end time (earlier cases train, later cases test). The value is the fraction used for training — the default `0.7` means 70 % training / 30 % test. Press `Enter` to keep the default.

3.7 Step 5: ML Algorithm and Hyperparameters

The algorithm menu depends on the prediction target. The two classification targets (process outcome, future deviations) offer classifiers; remaining time offers regressors.

3.7.1 Classification algorithms (process outcome / future deviations)

```
Which classification algorithm do you want to apply?
(1) Decision Tree
(2) Logistic Regression
(3) Gradient Boosting
Select option (1-3):
```

All three are fully implemented and support binary and multi-class targets.

3.7.2 Regression algorithms (remaining time)

```
Which regression algorithm do you want to apply?
(1) Decision Tree Regressor
(2) Random Forest Regressor
(3) Gradient Boosting Regressor
Select option (1-3):
```

3.7.3 Hyperparameters

After selecting an algorithm:

```
Do you want to configure the parameters yourself? (y/N):
```

Answer `n` (the default) to use sensible defaults, or `y` to set the parameters relevant to your chosen algorithm. Press `Enter` on any field to keep its default value.

Decision Tree (classification):

```
Criterion (gini/entropy, default gini):
Max depth (int, default 4):
Min samples split (int, default 2):
Min samples leaf (int, default 1):
```

Logistic Regression:

```
Inverse regularization strength C (float, default 1.0):
```

Gradient Boosting (classification):

```
Number of estimators (int, default 100):
Max depth (int, default 3):
```

Decision Tree Regressor (remaining time):

```
Max depth (int, default 4):
Min samples split (int, default 2):
Min samples leaf (int, default 1):
```

Random Forest / Gradient Boosting Regressor (remaining time):

```
Number of estimators (int, default 100):
Max depth (int, blank = default):
```

3.8 Step 6: Decision Tree Visualization and Refinement

This step only appears when a single Decision Tree was selected — either the Decision Tree classifier or the Decision Tree Regressor. We recommend, to visualize the trained tree at least once to understand how the decision tree looks like with the default parameters and specified hyperparameters. After each visualization, you have the option to reconfigure the hyperparameters and retrain the model, such that you can interatively refine the tree structure and inspect the changes after each iteration.

After the initial training, the tool asks:

```
Do you want the decision tree saved (s), displayed (d)
or not shown (N)? (s/d/N):
```

- **s — Save:** provide a file path when asked:

  ```
  Enter the path to save the decision tree (e.g., tree.png):
  ```

  The tree is rendered with feature names and class labels and saved as an image file.

- **d — Display:** the tree is opened in an interactive Matplotlib window.
- **N — Skip:** no visualization is produced.

Immediately after the visualization choice, the tool asks:

```
Do you want to reconfigure the hyperparameters and retrain? (y/N):
```

If you answer `y`, the Decision Tree parameter prompt from Step 5 is shown again. The model is retrained with the new settings and the visualization prompt is repeated, allowing you to inspect the new tree before deciding again. This loop continues until you answer `n`.

3.9 Step 7: Evaluation Results

Once the final model is confirmed, evaluation runs automatically. What is printed depends on the target and on whether prefixes vary.

Scalar metrics (all targets). A block of scalar metrics is always printed, labelled with its scope (full traces, or pooled over the prefix lengths used). For the classification targets these are accuracy, precision, recall, F1-score and ROC-AUC; for remaining time these are regression metrics (e.g. error measures).

Confusion matrix and per-class breakdown (process outcome only). For the process-outcome target the tool additionally prints:

1. A **confusion matrix** as a text table, with rows representing actual class labels and columns representing predicted class labels.

2. A **per-class accuracy breakdown** listing, for each class, the percentage of cases that were correctly predicted and the percentage that were not.

Metrics by prefix length (varying prefixes only). If you chose the *varying* prefix mode (so at least two distinct prefix lengths are present), the tool also prints a table of metrics broken down by prefix length, letting you see how predictive performance changes as more of the case is observed. For classification it then offers an interactive loop to inspect a single length in detail:

```
Enter a prefix length to see its specific confusion matrix
(or 'N' to skip):
```

Enter a length to print that length's confusion matrix and per-class breakdown, or `N` to skip. Finally it offers to save a performance-by-prefix-length chart:

```
Do you want to save a visualization of the performance
per prefix-length? (y/N):
Enter filename (default: by_len_vis.png):
```

3.10 Step 8: Saving Evaluation Visuals

This prompt only appears for the process-outcome target.

```
Save confusion matrix and accuracy chart? (y/N):
```

If you answer `y`:

```
Folder to save in (press Enter for Downloads):
```

Press `Enter` to save to your system's `Downloads` folder, or type a custom folder path. The tool creates a timestamped subfolder (e.g. `conformance_results_2026-05-15_14-30-00`) inside the chosen location and saves two image files:

- `confusion_matrix.png` — a color-coded heatmap of the confusion matrix.
- `per_class_accuracy.png` — a stacked bar chart with green segments (correct predictions) and red segments (incorrect predictions) per class.

3.11 Step 9: Predictions on New Data

```
Do you want to perform predictions on new data? (y/N):
```

Not yet implemented. Answering `y` prompts for a file path containing trace prefixes, but no predictions are computed; the program continues to the runtime report regardless of the answer.

3.12 Step 10: Runtime Report

After the prediction steps, the tool prints the runtime of each pipeline step and the peak memory used, followed by the total processing time. It then offers two optional reports.

A pie chart of the processing times per step:

```
Do you want to visualize the processing times as a pie chart? (y/N):
Do you want to save the pie chart (s) or directly display it (d)? (s/d):
```

Choosing `s` asks for an output path (default `runtime_dist.png`); choosing `d` opens the chart in a window.

A machine-readable JSON report:

```
Do you want to save the runtime and memory measurements as JSON? (y/N):
Enter filename (default: report.json):
```

The resulting file contains the per-step runtimes, the peak memory in MB, and the total processing time.

3.13 Known Limitations

Most features from earlier sprints are now fully functional. The following limitations remain worth keeping in mind:

- **Genetic Miner** — discovery is very slow on larger logs (around 19 minutes on the 1050-case Sepsis log, versus seconds for Inductive/Alpha), and the discovered net may not be "easy sound". In that case any alignment-based path (future-deviation labels, diagnostics or hybrid features) fails with a caught `[EXTRACTION ERROR]` and the program exits cleanly. Prefer the Inductive or Alpha Miner for alignment-based targets.
- **Future deviations on short prefixes** — with a short fixed prefix and `T=0`, all training prefixes may be non-deviant, leaving a single training class. The training step is not wrapped in error handling, so this can terminate the program with a traceback instead of a friendly message. Use a longer prefix, the varying prefix mode, or full traces to avoid it.
- **Prediction on new data** (Step 9) — the prompt appears, but no prediction logic is executed yet.
- **CSV log format** — only supported if the file uses the exact column names `case:concept:name`, `concept:name`, and `time:timestamp`.
- **Multi-outcome cases** — for the process-outcome target, a case containing more than one (or none) of the specified outcome activities receives the label `uncertain_outcome` rather than a specific class.